

RÉPUBLIQUE DU CAMEROUN
REPUBLIC OF CAMEROON
Peace – Work – Fatherland

UNIVERSITÉ DE DSCHANG
UNIVERSITY OF DSCHANG
Scholae Thesaurus Dschangensis Ibi Cordum

BP 96, Dschang (Cameroun)
Tél./Fax : (237) 233 45 13 81
Website : <http://www.univ-dschang.org>
E-mail : udsrectorat@univ-dschang.org



FACULTÉ DES SCIENCES
FACULTY OF SCIENCES

Département de Mathématiques et
Informatique
*Department of Mathematics and
Computer Science*

BP 67, Dschang (Cameroun)
Tél. : (237) 233 45 17 35
Website : <http://fs.univ-dschang.org>
E-mail : dept-math-info@univ-dschang.org

SUPPORT VECTOR MACHINES, RANDOM FORESTS & EXTREME GRADIENT BOOSTING

Niveau : **MASTER 2**

Option : **Intelligence Artificielle**

Sous la supervision de :

Pr.KENGNE TCHENDJI Vianney
Maître de Conférences, Université de Dschang

Année Académique : 2025 - 2026

Membres du groupe 3

Nom	Matricule
LEKANE KENFACK VERGEZ	CM-UDS-21SCI0569
MBOUMEU YOUNBI AXEL AUBIN	CM-UDS-21SCI0589
MOUNCHIKPOU MEFIRE HAMED KAMAL	CM-UDS-21SCI0601
NJIH ASONYU SENGE	CM-UDS-21SCI0625

- 1 Introduction
- 2 SVM
- 3 Bagging vs Boosting
- 4 Random Forests
- 5 XGBoost
- 6 Conclusion Générale

- 1 Introduction
- 2 SVM
- 3 Bagging vs Boosting
- 4 Random Forests
- 5 XGBoost
- 6 Conclusion Générale

Dans le domaine du machine learning supervisé, plusieurs algorithmes se distinguent par leur efficacité pour la classification et la régression. Parmi eux, les **SVM**, les **Random Forests** et **XGBoost** occupent une place centrale grâce à leur robustesse et leurs performances élevées sur des données complexes.

Ces modèles reposent sur des approches différentes :

- Les **SVM** maximisent la marge de séparation entre les classes ;
- Les **Random Forests** combinent plusieurs arbres de décision pour améliorer la généralisation ;
- **XGBoost** utilise le boosting pour corriger progressivement les erreurs.

- **Apprentissage automatique (Machine Learning)** : domaine de l'IA permettant aux systèmes d'apprendre à partir des données sans être explicitement programmés.
- **Classification** : tâche supervisée consistant à prédire une étiquette discrète (ex : spam / non-spam, malade / sain).
- **Régression** : tâche supervisée visant à prédire une valeur continue (ex : prix d'une maison, température).

- 1 Introduction
- 2 SVM**
- 3 Bagging vs Boosting
- 4 Random Forests
- 5 XGBoost
- 6 Conclusion Générale

Définition

Algorithme d'apprentissage supervisé introduit par **Vapnik et al. (1992)** pour la **classification** et la **régression**.

Idée centrale :

- Trouver l'**hyperplan optimal** séparant les classes
- Maximiser la **marge** entre les deux classes
- Fondé sur l'**optimisation convexe**

Points forts :

- Robuste en grande dimension
- Résistant au surapprentissage
- Applicable aux données non linéaires

Support Vector Machines (SVM)

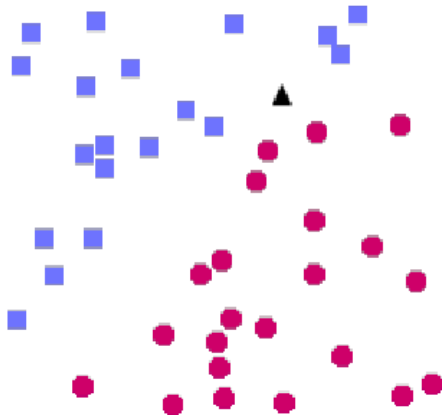


Figure 1 – L'objet à classer (le triangle noir) est-il un rond rouge ou bien un carré bleu ?

Support Vector Machines (SVM)

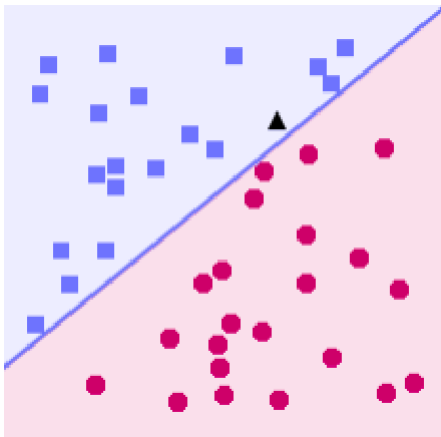


Figure 2 – Notre SVM, muni des donn'ees d'entraînement (les carr'es bleus et les ronds rouges d'ejà indiqu'es comme tels par l'utilisateur), a tranch'e : le triangle noir est en fait un carre bleu

Pour $x \in \mathbb{R}^n$, le SVM cherche un **hyperplan affine** :

$$h(x) = \mathbf{w}^\top x + b$$

- $\mathbf{w}$: vecteur de poids b : biais
- Classification : $y = +1$ si $h(x) \geq 0$, $y = -1$ sinon

Contraintes d'apprentissage

$$y_i (\langle \mathbf{w}, x_i \rangle + b) \geq 1 \quad \forall i$$

Objectif : maximiser la marge

$$\text{Marge} = \frac{2}{\|\mathbf{w}\|} \quad \Rightarrow \quad \text{minimiser } \|\mathbf{w}\|$$

Maximisation de la marge

- H : hyperplan séparateur
- H^+ : hyperplan longeant les positifs
- H^- : hyperplan longeant les négatifs
- La **marge** = distance entre H^+ et H^-

Distance point-hyperplan

$$d = \frac{|\mathbf{w}^\top x_k + b|}{\|\mathbf{w}\|}$$

Les **vecteurs de support** sont les points les plus proches de H .

Résultat clé

$$\text{Largeur de la marge} = \frac{2}{\|\mathbf{w}\|}$$

$\Rightarrow$ maximiser la marge = minimiser $\|\mathbf{w}\|$

Maximisation de la marge

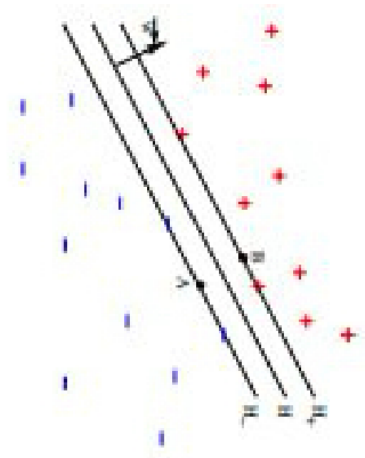


Figure 3 – Schéma illustratif : groupe H^+ et H^-

Formulation Lagrangienne des SVM

Problème primal : minimiser $\frac{1}{2}\|\mathbf{w}\|^2$ sous contraintes $y_i(\mathbf{w}^\top x_i + b) \geq 1$

On introduit les **multiplicateurs de Lagrange** $\alpha_i \geq 0$:

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w}^\top x_i + b) - 1]$$

Conditions KKT :

$$\mathbf{w} = \sum_i \alpha_i y_i x_i, \quad \sum_i \alpha_i y_i = 0$$

$\Rightarrow$ **Problème dual** en α uniquement

Propriété clé

Seuls les points avec $\alpha_i > 0$ contribuent à $\mathbf{w}$:
ce sont les **vecteurs de support**.

La fonction de décision devient :

$$h(x) = \sum_i \alpha_i y_i K(x_i, x) + b$$

SVM non linéaires : Plongée en dimension supérieure

Problème : Certaines données **ne sont pas linéairement séparables** dans l'espace d'origine (ex : cercles concentriques, problème XOR).

Solution : Projeter les données dans un espace de **dimension supérieure** E' via une fonction ϕ :

$$\phi : E \rightarrow E', \quad x \mapsto \phi(x)$$

- Dans E' , les données deviennent linéairement séparables
- Le SVM trouve un hyperplan dans E'
- Ce plan correspond à une **frontière non linéaire** dans E

Problème pratique

Plus la dimension de E' est grande, plus les calculs sont **coûteux**.

⇒ Le **Kernel Trick** résout ce problème !

SVM non linéaires : Plongée en dimension supérieure



Figure 4 – Données d'entraînement non linéairement séparables

SVM non linéaires : Plongée en dimension supérieure

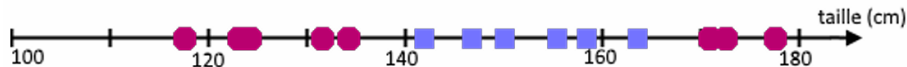


Figure 5 – Illustration : données sur une droite

SVM non linéaires : Plongée en dimension supérieure

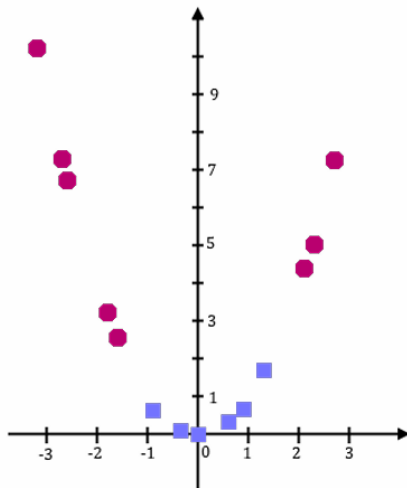


Figure 6 – Données d'entraînement après projection dans l'espace R^2

L'Astuce du Noyau (Kernel Trick)

Idée centrale : Les algorithmes n'ont besoin que des **produits scalaires** $\phi(x_i)^\top \phi(x_j)$.

On définit une **fonction noyau** : $K(x, x') = \phi(x)^\top \phi(x')$

Principaux noyaux :

- **Polynomial** : $K = (\alpha x^\top x' + \lambda)^d$
- **Gaussien (RBF)** :
$$K = \exp\left(-\frac{\|x-x'\|^2}{2\sigma^2}\right)$$
- **Laplacien** : $K = \exp\left(-\frac{\|x-x'\|}{\sigma}\right)$
- **Rationnel quadratique**

Avantage

Calcul en $\mathcal{O}(n)$ au lieu de $\mathcal{O}(n^2)$ pour le produit dans E' .

Pas besoin de connaître E' ni ϕ explicitement !

Types de noyaux

Noyau	Formule	Usage
Linéaire	$x^\top x'$	Données séparables
Polynomial	$(\alpha x^\top x' + \lambda)^d$	Écriture, NLP
Gaussien (RBF)	$\exp\left(-\frac{\ x-x'\ ^2}{2\sigma^2}\right)$	Usage général
Laplacien	$\exp\left(-\frac{\ x-x'\ }{\sigma}\right)$	Bio-informatique
Rat. Quadratique	$1 - \frac{\ x-x'\ ^2}{\ x-x'\ ^2 + \sigma}$	Recommandation

Condition de Mercer

Une fonction K est un noyau valide si et seulement si elle est **continue, symétrique et semi-définie positive** :
$$\sum_{i,j} c_i c_j K(x_i, x_j) \geq 0$$

Hyperparamètres clés des SVM

Paramètre C (régularisation)

Contrôle le **compromis biais-variance** :

- C grand $\Rightarrow$ marge étroite, peu d'erreurs tolérées $\Rightarrow$ surapprentissage
- C petit $\Rightarrow$ marge large, plus d'erreurs tolérées $\Rightarrow$ sous-apprentissage

Paramètre σ / γ (noyau RBF)

Contrôle la **portée d'influence** de chaque point :

- σ petit $\Rightarrow$ frontière très locale
- σ grand $\Rightarrow$ frontière lissée

Paramètre d (noyau polynomial)

Degré du polynôme :

plus d est grand, plus la frontière est complexe.

Stratégie de réglage

Utiliser la **validation croisée** (grid search) pour trouver la combinaison (C, σ) optimale.

Mauvais choix $\Rightarrow$ dégradation sévère des performances.

Avantages

- Efficace en **grande dimension**
- Robuste au **surapprentissage**
- Fonctionne avec peu de données
- Frontières complexes via noyaux
- Optimum **global** garanti

Limites

- Sensible au choix des **hyperparamètres** (C , σ)
- Coûteux pour de **grands jeux** de données
- Choix du noyau non trivial
- Moins interprétable que les arbres

Message clé

Les SVM cherchent l'**hyperplan à marge maximale**. Pour les données non linéaires, le **Kernel Trick** permet de travailler implicitement dans un espace de grande dimension, sans en calculer explicitement les coordonnées.

- 1 Introduction
- 2 SVM
- 3 Bagging vs Boosting**
- 4 Random Forests
- 5 XGBoost
- 6 Conclusion Générale

Concepts clés : Bagging vs Boosting

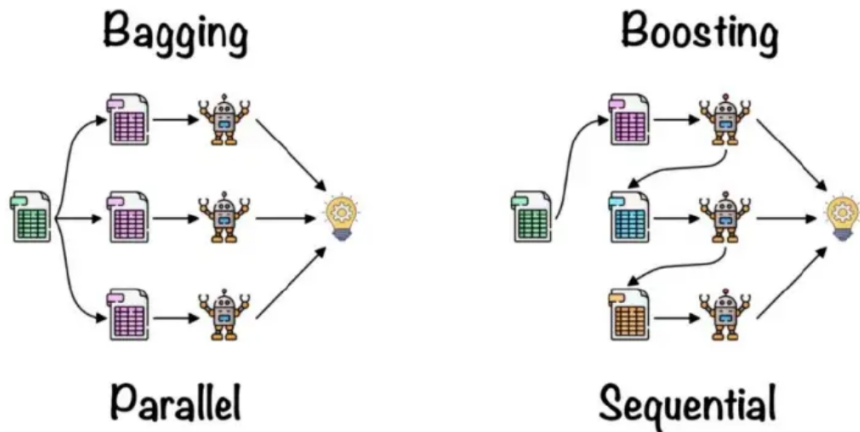


Figure 7 – Bagging (parallèle, réduit la variance) vs Boosting (séquentiel, réduit le biais).

- 1 Introduction
- 2 SVM
- 3 Bagging vs Boosting
- 4 Random Forests**
- 5 XGBoost
- 6 Conclusion Générale

- Les arbres de décision simples sont sujets au **surapprentissage (overfitting)**.
- Les **Random Forests** combinent plusieurs arbres pour un modèle plus **robuste et précis**.
- Principe clé : la *sagesse des foules* — agréger les prédictions de nombreux arbres entraînés sur des sous-échantillons différents.
- Contrairement aux SVM et à XGBoost, les Random Forests construisent des arbres **indépendants et parallèles**.

Fonctionnement du Random Forest

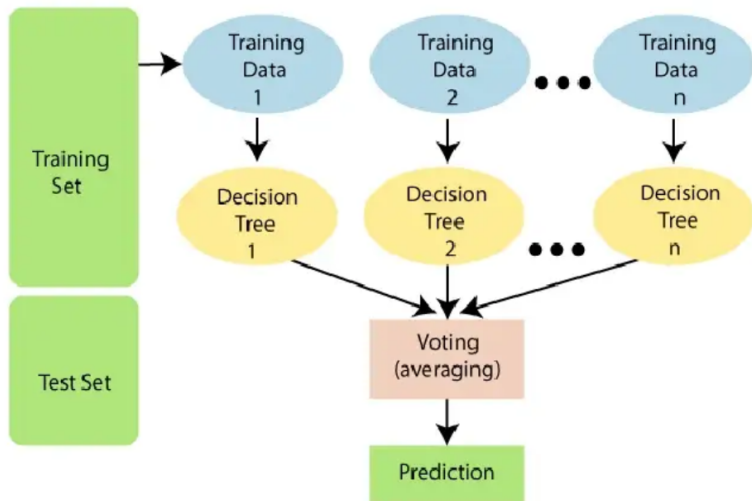


Figure 8 – Principe de fonctionnement du Random Forest

Rappel : Arbres de décision

- Un **arbre de décision** est un modèle hiérarchique basé sur une succession de questions binaires (oui/non).
- Il est composé de :
 - **Noeuds internes** : décisions sur les variables d'entrée ;
 - **Branches** : résultats possibles des décisions ;
 - **Feuilles** : prédictions finales.
- Exemple : « Âge > 30 ? », puis « Revenu > 50 000 ? » pour prédire un achat.

Rappel visuel : Arbre de décision

Exemple d'Arbre de Decision - Prediction d'achat

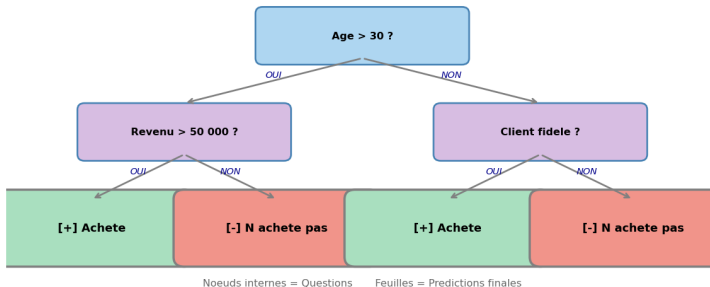


Figure 9 – Exemple d'arbre de décision pour la prédiction d'un achat.

- L'arbre se construit de manière **récursive** en choisissant à chaque étape la meilleure division.
- Sans contraintes, l'arbre peut **surapprendre** :
 - Excellente performance sur les données d'entraînement ;
 - Mauvaise généralisation sur de nouvelles données.
- Les arbres ont **faible biais** mais **forte variance** → motivation des Random Forests.

- Deux sources principales de hasard :
 - L'**échantillonnage bootstrap** des données ;
 - La **sélection aléatoire des caractéristiques**.
- Ces mécanismes permettent de construire un ensemble d'arbres **diversifiés et moins corrélés**.

Étape 1 : Échantillonnage Bootstrap

- Pour chaque arbre, on génère un échantillon aléatoire **avec remise**.
- Certaines observations peuvent apparaître plusieurs fois, d'autres peuvent être absentes.
- En moyenne, chaque échantillon contient environ **63,2%** d'observations uniques.
- Cela crée des jeux de données légèrement différents pour entraîner chaque arbre.

Étape 2 : Sélection aléatoire des caractéristiques

- À chaque nœud, on sélectionne aléatoirement un sous-ensemble de caractéristiques ($m < p$).
- La meilleure division est choisie **uniquement parmi ces caractéristiques**.
- Objectif : réduire la corrélation entre les arbres et augmenter la **diversité** du modèle.

Étapes 3 et 4 : Croissance et Agrégation

- **Croissance des arbres :**

- Chaque arbre est développé jusqu'à sa profondeur maximale (sans élagage).

- **Agrégation des prédictions :**

- Classification : **vote majoritaire** des arbres ;
- Régression : **moyenne** des prédictions.

Hyperparamètre clé : Nombre d'arbres

- **n_estimators (ntree)** : nombre d'arbres dans la forêt.
- Valeurs typiques : 100 à 1000 arbres.
- Effets :
 - Plus d'arbres $\rightarrow$ variance réduite et prédictions plus stables ;
 - Temps de calcul plus élevé.
- Les gains deviennent faibles après un certain seuil (plateau de performance).
- Contrairement au boosting, ajouter des arbres ne provoque généralement **pas de surapprentissage**.

Hyperparamètre clé : max_features

- Contrôle le nombre de caractéristiques testées à chaque division.
- Valeurs par défaut :
 - Classification : $\sqrt{p}$
 - Régression : $p/3$
- Effets :
 - Valeurs faibles $\rightarrow$ arbres plus diversifiés mais plus faibles ;
 - Valeurs élevées $\rightarrow$ arbres plus puissants mais plus corrélés.
- C'est l'hyperparamètre le **plus important** à ajuster.

- **Profondeur des arbres :**
 - `max_depth`, `min_samples_split`, `min_samples_leaf`
- **Taille bootstrap (`max_samples`) :**
 - Fraction des données utilisée pour chaque arbre.
- **Stratégie pratique :**
 - 1 Commencer avec les paramètres par défaut ;
 - 2 Ajuster d'abord `n_estimators` ;
 - 3 Ajuster ensuite `max_features` (le plus influent) ;
 - 4 Utiliser l'erreur OOB pour une évaluation rapide.

Estimation de l'erreur Out-of-Bag (OOB) — Illustration

Echantillonnage Bootstrap et donnees Out-of-Bag (OOB)

Dataset original (n=10 observations)



Arbre 1 (Bootstrap):



Arbre 2 (Bootstrap):



Arbre 3 (Bootstrap):



Bleu = utilise (bootstrap) Rouge/OOB = non vu = ~ 36.8% des donnees

L'erreur OOB est calculee sur ces obs. -> validation integree, sans jeu de test separee !

Figure 10 – Schéma du mécanisme bootstrap et des données OOB par arbre.

Fonctionnement de l'erreur Out-of-Bag (OOB)

- Lors du bootstrap, environ **36,8%** des observations ne sont pas utilisées pour entraîner un arbre donné : ce sont les **échantillons Out-of-Bag (OOB)**.
- Pour chaque observation :
 - ➊ Identifier les arbres qui ne l'ont pas vue (arbres OOB).
 - ➋ Faire la prédiction uniquement avec ces arbres.
 - ➌ Comparer la prédiction à la vraie valeur.
- L'erreur OOB = moyenne des erreurs sur toutes les observations.
Avantage : validation intégrée sans jeu de test séparé.

Avantages :

- Estimation **non biaisée** de l'erreur de généralisation
- Aucun coût d'entraînement supplémentaire
- Permet de choisir le nombre d'arbres optimal

Limites :

- Peut être légèrement **optimiste** comparée à un test indépendant
- Moins fiable sur de **petits jeux de données**

Formule clé :

$$P(\text{non sélectionnée}) = \left(1 - \frac{1}{n}\right)^n \rightarrow \frac{1}{e} \approx 0,368$$

Importance des variables — Illustration

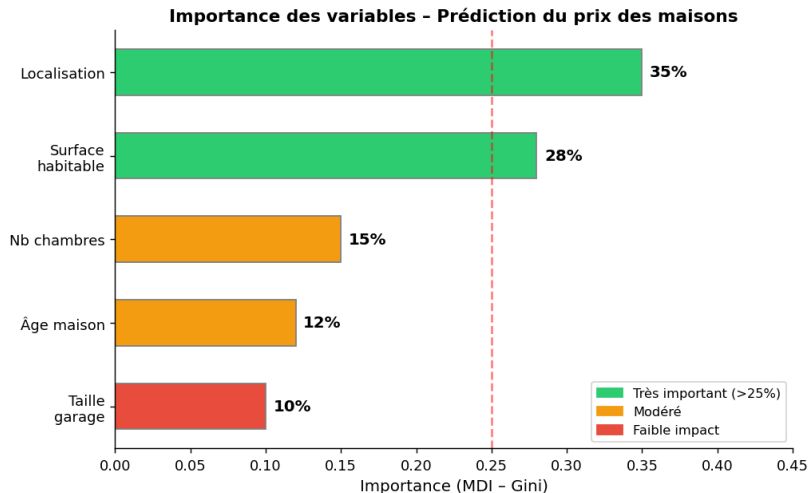


Figure 11 – Importance MDI (Gini) des variables pour la prédiction du prix des maisons.

Importance des variables : MDI (Gini Importance)

- La **Diminution Moyenne de l'Impureté (MDI)** mesure combien une variable réduit l'impureté des nœuds.
- Calcul : sommer la réduction d'impureté pondérée sur tous les arbres et normaliser.
- **Limites :**
 - Biais vers les variables à forte cardinalité
 - Peut surestimer l'importance si les variables sont corrélées

Avantages des Random Forests

- Robustesse au surapprentissage
- Prétraitement minimal des données
- Gestion des valeurs manquantes
- Capture naturelle des relations non linéaires
- Parallélisation et scalabilité
- Validation intégrée (erreur OOB)
- Importance des variables interprétable
- Bonnes performances avec paramètres par défaut
- Polyvalence (classification, régression, etc.)

- Difficulté d'interprétation
- Besoins élevés en mémoire
- Vitesse de prédiction plus lente
- Moins performant sur données hautement dimensionnelles
- Pas d'extrapolation en régression
- Importance des variables biaisée (MDI)
- Pas toujours le meilleur choix face au boosting ou deep learning

- Les Random Forests sont des modèles puissants et polyvalents basés sur le **bagging**
- Robustesse au surapprentissage et gestion des relations non linéaires
- Validation intégrée via l'**erreur Out-of-Bag (OOB)**
- Adaptées à la classification et à la régression
- Limites : interprétation difficile et coût mémoire élevé
- XGBoost adopte une approche **boosting** avec construction séquentielle des arbres

- 1 Introduction
- 2 SVM
- 3 Bagging vs Boosting
- 4 Random Forests
- 5 XGBoost**
- 6 Conclusion Générale

- **XGBoost** (eXtreme Gradient Boosting) : algorithme avancé de gradient boosting
- Introduit par Chen & Guestrin (2016)
- Optimisé, scalable et performant pour classification et régression
- Large succès en compétitions Kaggle et applications industrielles
- Utilise le **boosting** (construction séquentielle) pour réduire le biais
- Alternative performante aux méthodes basées sur le bagging (Random Forest)

Points clés qui distinguent XGBoost du Gradient Boosting classique :

- **Régularisation** intégrée
- Conception d'un **arbre de régression unique** optimisé
- **Algorithme glouton approximatif** pour le choix des coupures
- **Esquisse de quantiles pondérés** (weighted quantile sketch)
- Intégration de la parcimonie des données (**sparsity-aware split finding**)
- **Apprentissage parallèle** au niveau des nœuds
- **Accès optimisé au cache** du processeur (cache-aware access)
- **Calculs hors mémoire par blocs** (out-of-core computation)

Principe du Gradient Boosting

- Apprentissage **itératif et séquentiel**
- Chaque nouvel arbre apprend les **erreurs résiduelles** du modèle précédent
- Construction progressive du modèle global
- Interprétation : descente de gradient dans l'espace des fonctions
- Objectif : minimisation incrémentale de la **fonction de perte**

Principe du Gradient Boosting — Illustration

Principe du Gradient Boosting - Apprentissage des résidus

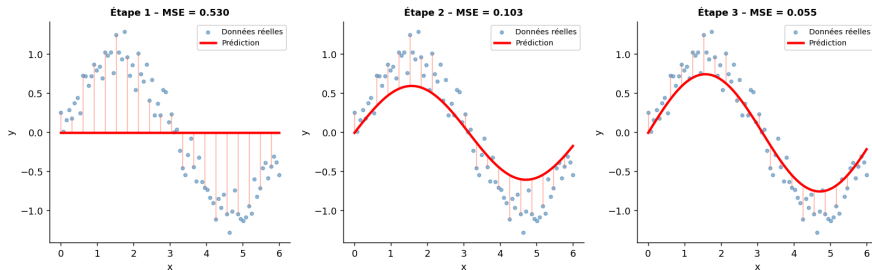
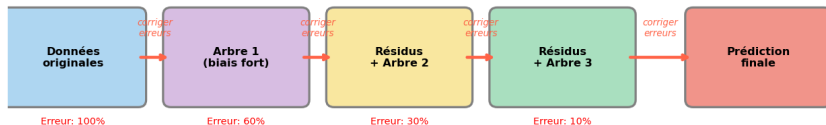


Figure 12 – Chaque arbre réduit progressivement les résidus du modèle précédent.

XGBoost : Construction séquentielle

XGBoost - Construction séquentielle par correction des erreurs



✂ Chaque arbre apprend sur les résidus du précédent → Réduction progressive du biais

Figure 13 – XGBoost corrige progressivement les erreurs des modèles précédents.

XGBoost pour la Régression : Initialisation

Jeu de données (Médicaments) :

Dosage (mg)	Efficacité
10	-10
20	7
25	8
35	-7

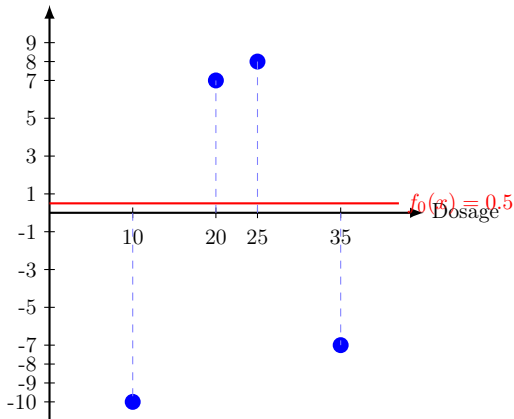
Prédiction initiale $f_0(x)$

Valeur par défaut : **0.5**.

Pourquoi 0.5 ?

Dans XGBoost, le score de base (`base_score`) est initialisé à 0.5 par défaut, un héritage de la probabilité. En régression, ce "décalage" n'est pas pénalisant : les arbres de gradient vont très vite corriger cette valeur initiale via l'apprentissage itératif des résidus.

Efficacité (y)



Calcul des Résidus (r_i) :

$$r_i = y_i - f_0(x_i)$$

(Efficacité réelle - prédiction)

Dosage	Effic.	Résidu (r_i)
10	-10	-10.5
20	7	6.5
25	8	7.5
35	-7	-7.5

Exemple pour dosage 10 :

$$r_1 = -10 - 0.5 = -10.5$$

Métriques de construction

1. Similarity Score (SS) : $SS = \frac{(\sum \text{Résidus})^2}{\text{Nb. Résidus} + \lambda}$

Évalue la pureté du nœud.

2. Gain : $\text{Gain} = SS_{\text{gauche}} + SS_{\text{droit}} - SS_{\text{racine}} - \gamma$
Mesure l'amélioration apportée par l'embranchement.

3. Hyperparamètres de Régularisation :

- λ (Lambda) : Régularisation L2. Diminue l'influence des observations individuelles.
- γ (Gamma) : Seuil minimal de Gain requis pour diviser.

Score de la Racine (avec $\lambda = 0$) :

Somme des résidus = -4. Nombre = 4.

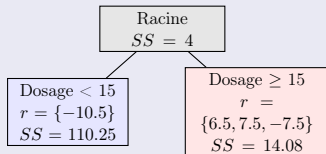
$$\rightarrow SS_{\text{racine}} = \frac{(-4)^2}{4+0} = \frac{16}{4} = 4.$$

XGBoost : Évaluation des Splits (Arbre 1)

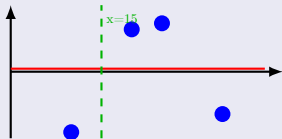
On évalue les moyennes entre dosages consécutifs : 15, 22.5, 30.

$(\lambda = 0, \gamma = 0)$

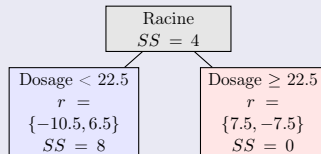
Split 1 : Dosage < 15



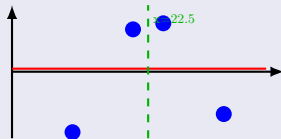
$$\text{Gain} = 110.25 + 14.08 - 4 = 120.33$$



Split 2 : Dosage < 22.5

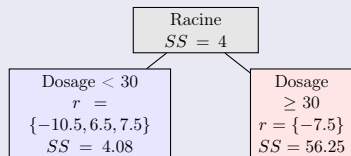


$$\text{Gain} = 8 + 0 - 4 = 4.00$$

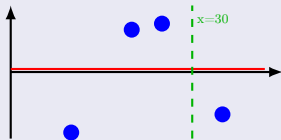


XGBoost : Split 3 et Conclusion de l'Arbre 1

Split 3 : Dosage < 30



$$\text{Gain} = 4.08 + 56.25 - 4 = \mathbf{56.33}$$



Comparaison des Splits ($\lambda = 0, \gamma = 0$)

- Gain (Split < 15) : **120.33**
- Gain (< 22.5) : 4.00 | Gain (< 30) : 56.33

Verdict de l'algorithme

L'approche gloutonne (Greedy Algorithm) sélectionne le split **maximisant le Gain**.
 $\Rightarrow$ On isole la valeur très négative en coupant à **Dosage < 15**.

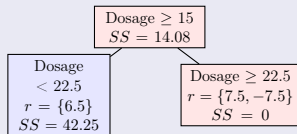
Le processus se répète ensuite de façon récursive sur les nœuds enfants.

XGBoost : Poursuite de la construction (Nœud Droit)

Le nœud de droite ($\text{Dosage} \geq 15$) contient les résidus $\{6.5, 7.5, -7.5\}$.

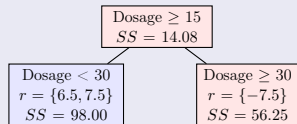
Coupures possibles : 22.5 et 30.

Nouvelle coupure : < 22.5



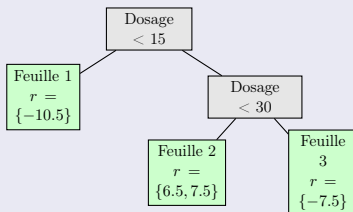
$$\text{Gain} = 42.25 + 0 - 14.08 = \mathbf{28.17}$$

Nouvelle coupure : < 30



$$\text{Gain} = 98.00 + 56.25 - 14.08 = \mathbf{140.17}$$

Arbre final (Profondeur = 2)



Valeurs de Sortie (Output)

Pour chaque feuille, on calcule la valeur de sortie : $\text{Output} = \frac{\sum \text{Résidus}}{\text{Nb. Résidus} + \lambda}$

- F1 : $-10.5/1 = \mathbf{-10.5}$
- F2 : $(6.5 + 7.5)/2 = 14/2 = \mathbf{7}$
- F3 : $-7.5/1 = \mathbf{-7.5}$

XGBoost : Régularisation par λ et γ

1. Effet de Lambda (λ) sur le Score de Similarité

$SS = \frac{(\sum \text{Résidus})^2}{N + \lambda}$ (N = nombre d'obs.). Si $\lambda > 0$, le SS baisse.

Réduction inversement proportionnelle à N :

- **Peu de données** ($N = 1$) : λ domine le dénominateur $\rightarrow$ baisse **drastique** du SS .
- **Beaucoup de données** ($N = 100$) : l'ajout de λ devient négligeable.

$\Rightarrow$ **Effet** : λ pénalise le surapprentissage en élaguant les feuilles très isolées.

Exemple comparatif (SS avec $\lambda = 1$)

Feuille isolée ($N = 1$, $R=7.5$) :

- $\lambda = 0 \rightarrow SS = 7.5^2/1 = \mathbf{56.25}$
- $\lambda = 1 \rightarrow SS = 7.5^2/2 = \mathbf{28.12}$ (-50%)

Feuille groupée ($N = 2$, $R=6.5, 7.5$) :

- $\lambda = 0 \rightarrow SS = 14^2/2 = \mathbf{98.00}$
- $\lambda = 1 \rightarrow SS = 14^2/3 = \mathbf{65.33}$ (-33%)

2. Effet de Gamma (γ)

$Gain = SS_{gauche} + SS_{droit} - SS_{racine} - \gamma$

Élagage de bas en haut (Post-Pruning) :

- Si **Gain** < 0 , la branche est supprimée.
- γ représente un seuil de "péage" minimal.

$\Rightarrow$ **Effet** : γ exige une amélioration **substantielle**, évitant de conserver les ajustements microscopiques du modèle.

XGBoost : Application du Learning Rate (η)

Le Taux d'Apprentissage (η) / Shrinkage

Pour éviter le surapprentissage, la sortie de l'arbre n'est pas ajoutée intégralement. On applique une technique de **Shrinkage** (réduction) en multipliant par un taux η (ex : 0.3).

$$f_1(x) = f_0(x) + \eta \times \text{Sortie Arbre}_1(x)$$

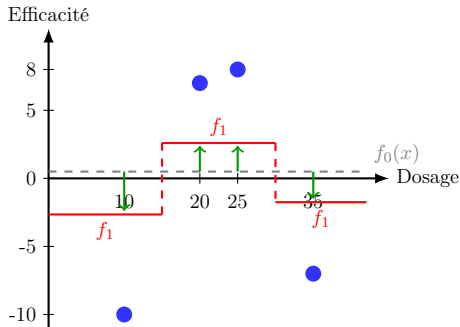
Calcul des nouvelles prédictions et résidus

Dosage	Sortie Arbre	Nouvelle Préd. (f_1)	Nouveau Résidu (r_i)
10	-10.5	$0.5 + 0.3(-10.5) = \mathbf{-2.65}$	$-10 - (-2.65) = \mathbf{-7.35}$
20	7	$0.5 + 0.3(7) = \mathbf{2.6}$	$7 - 2.6 = \mathbf{4.4}$
25	7	$0.5 + 0.3(7) = \mathbf{2.6}$	$8 - 2.6 = \mathbf{5.4}$
35	-7.5	$0.5 + 0.3(-7.5) = \mathbf{-1.75}$	$-7 - (-1.75) = \mathbf{-5.25}$

Observation : Tous les nouveaux résidus sont plus proches de 0 que les précédents (-10.5, 6.5, 7.5, -7.5). L'erreur globale a diminué !

XGBoost : Visualisation de $f_1(x)$ et Itérations

Un pas "prudent" vers la réalité ($\eta = 0.3$)



La suite de l'algorithme

L'erreur diminue progressivement ("booster").

Désormais, pour l'étape suivante :

- Construction de l'**Arbre 2**.
- Cible : les **nouveaux résidus** r_i qu'il nous reste à combler (-7.35, 4.4, 5.4, -5.25).

Le processus se répète itérativement ($f_2, f_3, \dots, f_M$) jusqu'à :

- Atteindre M arbres (`n_estimators`).
- Erreur quasi-nulle (*Early Stopping*).

XGBoost (Classification) : Initialisation et Log(Odds)

Jeu de données :

Dosage	Couleur	Efficacité (y)
2	Rouge	0 (Non)
8	Vert	1 (Oui)
12	Vert	1 (Oui)
18	Rouge	0 (Non)

Prédiction initiale p_0

Par défaut, on prédit $p_0 = 0.5$ pour tout le monde.

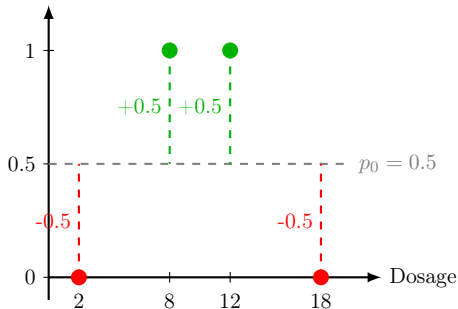
Cependant, XGBoost construit ses arbres avec des **Log(Odds)** (Log-cotes).

$$\text{Log(Odds)} = \log\left(\frac{p}{1-p}\right)$$

Si $p = 0.5$, l'initialisation vaut 0.

Résidus initiaux ($r_i = y_i - p_0$)

Probabilité (p)



Les pointillés montrent nos erreurs initiales (r_i).
XGBoost va s'en servir pour construire les arbres !

XGBoost (Classification) : Score de Similarité

Nouveau Score de Similarité (SS)

La formule s'adapte pour lier les probabilités (les résidus) et l'espace Log(Odds) des feuilles :

$$SS = \frac{(\sum \text{Résidus}_i)^2}{\sum \text{Cover}_i + \lambda}$$

Calcul de la Racine ($\lambda = 0$)

Pour le nœud racine (les 4 patients) :

- $\sum r_i = (-0.5) + 0.5 + 0.5 - 0.5 = 0$
- Numérateur : $0^2 = 0$.

$$\text{Score Racine} = \frac{0}{1.0 + 0} = 0$$

La notion de "Cover" (Couverture)

Remplace le nombre d'observations (N) par la **variance** des prédictions précédentes :

$$\text{Cover}_i = p_i \times (1 - p_i)$$

Ici, comme $p_0 = 0.5$ pour tous :

$$\text{Cover}_i = 0.5 \times 0.5 = \mathbf{0.25}$$

Le dénominateur total vaut donc :

$$\sum \text{Cover}_i = 0.25 \times 4 = \mathbf{1.0}$$

XGBoost : Évaluation des Splits (Coupure à 5)

Rappel : Gain

Gain = $SS_{gauche} + SS_{droit} - SS_{racine} - \gamma$. Testons notre premier split !

Split A : Dosage < 5

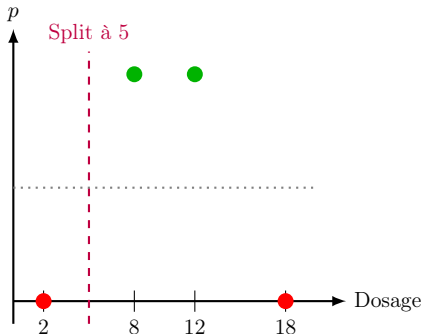
Gauche (Dosage 2) :

- $\sum r = -0.5$
- $SS_G = \frac{(-0.5)^2}{0.25} = 1.0$

Droit (Dosages 8, 12, 18) :

- $\sum r = 0.5 + 0.5 - 0.5 = 0.5$
- $SS_D = \frac{(0.5)^2}{0.25 \times 3} = 0.33$

$$\text{Gain} = 1.0 + 0.33 - 0 = 1.33$$



Isole la première valeur rouge. C'est un bon début !

XGBoost : Évaluation des Splits (Coupure à 10)

Split B : Dosage < 10

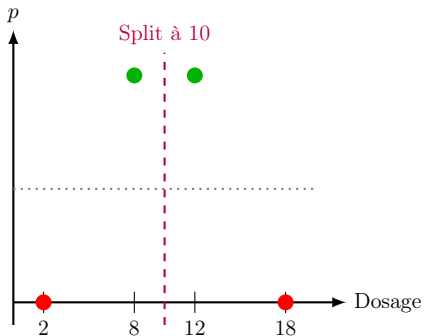
Gauche (Dosages 2, 8) :

- $\sum r = -0.5 + 0.5 = 0$
- $SS_G = \frac{0^2}{0.5} = 0$

Droit (Dosages 12, 18) :

- $\sum r = 0.5 - 0.5 = 0$
- $SS_D = \frac{0^2}{0.5} = 0$

$$\text{Gain} = 0 + 0 - 0 = 0$$



*Mélange un rouge et un vert de chaque côté. **Pire split possible !***

XGBoost : Évaluation des Splits (Coupure à 15)

Split C : Dosage < 15

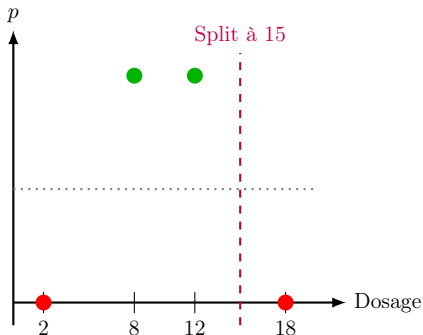
Gauche (Dosages 2, 8, 12) :

- $\sum r = -0.5 + 0.5 + 0.5 = 0.5$
- $SS_G = \frac{0.5^2}{0.75} = \mathbf{0.33}$

Droit (Dosage 18) :

- $\sum r = -0.5$
- $SS_D = \frac{(-0.5)^2}{0.25} = \mathbf{1.0}$

$$\text{Gain} = 0.33 + 1.0 - 0 = \mathbf{1.33}$$



Gain = 1.33. Ex aequo avec le Split A ! Pourquoi le choisir ?

Parce qu'il sépare la classe majoritairement positive (Gauche) de la donnée la plus extrême (Celle de droite).

XGBoost : Construction de l'Arbre 1 (Profondeur 2)

Poursuite sur le Nœud Gauche (Dosages 2, 8, 12)

On teste à nouveau une coupure. Testons **Split D : Dosage < 5** sur ce sous-groupe :

Évaluation du Split D

Gauche (Dosage 2) :

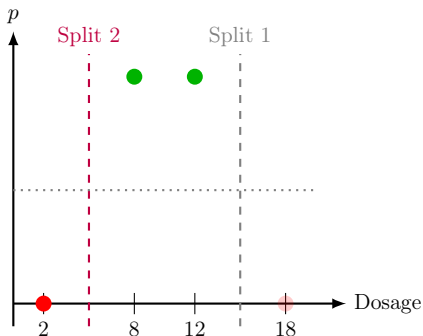
- $\sum r = -0.5$
- $SS_G = \frac{(-0.5)^2}{0.25} = 1.0$

Droit (Dosages 8, 12) :

- $\sum r = 0.5 + 0.5 = 1.0$
- $SS_D = \frac{(1.0)^2}{0.5} = 2.0$

Rappel $SS_{RacineGauche} = 0.33$

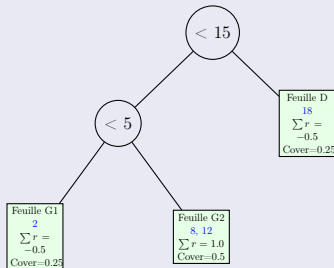
$$\text{Gain}_D = 1.0 + 2.0 - 0.33 = \mathbf{2.67}$$



Excellent Split (Gain = 2.67)! Il isole parfaitement le dernier point rouge des points verts.

XGBoost (Classification) : Valeur de Sortie

Arbre 1 Final (Profondeur 2)



Valeur de Sortie (Output Value)

Une fois l'arbre construit, on calcule sa valeur de sortie pour faire des prédictions.

La formule est la même que le Similarity Score, **sans le carré au numérateur** :

$$\text{Output} = \frac{\sum \text{Résidus}}{\sum \text{Cover} + \lambda}$$

Feuille G1 (Dosage 2) :

$$O_{G1} = \frac{-0.5}{0.25} = \mathbf{-2.0}$$

Feuille G2 (Dosages 8, 12) :

$$O_{G2} = \frac{1.0}{0.5} = \mathbf{2.0}$$

Feuille D (Dosage 18) :

XGBoost (Classification) : Mise à jour des Log-Odds

1. Log-Odds de l'étape 1 ($\eta = 0.3$)

On met à jour les Log-Odds initiaux (qui valaient 0) avec les sorties de l'arbre :

$$LO_1 = LO_0 + \eta \times \text{Output}$$

- **Feuilles G1 et D** (Points 2, 18) :

$$LO_1 = 0 + 0.3 \times (-2.0) = \mathbf{-0.6}$$

- **Feuille G2** (Points 8, 12) :

$$LO_1 = 0 + 0.3 \times (2.0) = \mathbf{0.6}$$

Pourquoi additionner en Log-Odds ?

L'addition directe de probabilités pourrait nous faire sortir de l'intervalle $[0, 1]$. L'addition dans l'espace Log-Odds (infini) garantit qu'après passage dans la fonction logistique, la probabilité restera **toujours** cohérente.

XGBoost (Classification) : Retour aux Probabilités

Conversion (p_1)

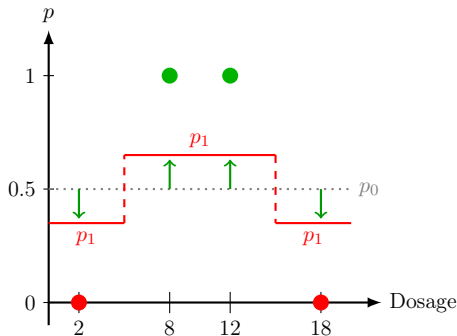
On utilise la **Fonction Logistique** :

$$p_1 = \frac{e^{LO_1}}{1 + e^{LO_1}}$$

- **F. G1/D** ($LO = -0.6$) :
 $p_1 \approx \mathbf{0.35}$
- **F. G2** ($LO = +0.6$) :
 $p_1 \approx \mathbf{0.65}$

Les probabilités ne sont plus bloquées à 0.5 ! Le modèle commence à se mouiller.

Mise à jour graphique



XGBoost (Classification) : Itérations Suivantes

Calcul des nouveaux résidus (r_1)

Répétition du processus : $r_1 = y_i - p_1$. L'erreur a globalement diminué!

Dosage	y_i	Ancien p_0 (r_0)	Nouveau p_1	Nouveau Résidu r_1
2 (R)	0	0.5 (-0.5)	0.35	$0 - 0.35 = \mathbf{-0.35}$
8 (V)	1	0.5 (+0.5)	0.65	$1 - 0.65 = \mathbf{+0.35}$
12 (V)	1	0.5 (+0.5)	0.65	$1 - 0.65 = \mathbf{+0.35}$
18 (R)	0	0.5 (-0.5)	0.35	$0 - 0.35 = \mathbf{-0.35}$

Construction de l'Arbre 2

- On construit maintenant un deuxième arbre.
- Son objectif est de prédire ces **nouveaux résidus** r_1 (± 0.35).
- Les "Covers" seront mis à jour avec les nouveaux p_1 .

Boucle Algorithmique

Le processus se répète ($p_2, p_3, \dots, p_M$) jusqu'à :

- Atteindre le nombre max d'arbres (**n_estimators**).
- Atteindre l'**Early Stopping** (les résidus ne diminuent plus).

Le Cœur de l'Optimisation

Pour éviter le surapprentissage tout en minimisant l'erreur, XGBoost minimise une fonction objectif composée :

$$\mathcal{L}(\phi) = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

- $\sum L(y_i, \hat{y}_i)$: Fonction de perte (Training Loss). Elle mesure à quel point le modèle est proche des données réelles.
- $\sum \Omega(f_k)$: Terme de régularisation. Il contrôle la complexité des arbres pour éviter le surapprentissage.

XGBoost : Le Terme de Régularisation $\Omega(f)$

Complexité d'un arbre

La complexité d'un arbre f est définie par sa structure (nombre de feuilles) et ses poids :

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 + \alpha \sum_{j=1}^T |w_j|$$

- γ (**Gamma**) : Pénalité sur le nombre de feuilles T . Permet l'élagage (Pruning).
- λ (**Lambda**) : Régularisation $L2$ sur les poids w_j (lissage, type Ridge).
- α (**Alpha**) : Régularisation $L1$ sur les poids w_j (parcimonie, type Lasso).

XGBoost : Approximation de Taylor (2nd Ordre)

Pour optimiser n'importe quelle fonction de perte L complexe, on utilise un développement de Taylor autour de la prédiction précédente $\hat{y}_i^{(t-1)}$:

$$L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) \approx L(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)$$

Gradient (g_i) et Hessienne (h_i)

Ces termes calculent la direction et la courbure de l'erreur :

$$g_i = \partial_{\hat{y}_i^{(t-1)}} L(y_i, \hat{y}_i^{(t-1)}) \quad , \quad h_i = \partial_{\hat{y}_i^{(t-1)}}^2 L(y_i, \hat{y}_i^{(t-1)})$$

Pourquoi le 2nd ordre ?

L'utilisation de la Hessienne (h_i) permet une convergence beaucoup plus rapide que la simple descente de gradient classique utilisée dans les GBM standards.

XGBoost : Calcul du Poids Optimal (Output Value)

En regroupant les données par feuille j , on cherche le poids w_j (ou "Output Value") qui minimise l'objectif local :

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right]$$

Solution Analytique

En dérivant cette équation (qui est une parabole) par rapport à w_j et en l'égalant à zéro, on trouve le minimum exact :

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

1. Cas de la Régression (Erreur Carrée)

Formule de perte : $L(y_i, \hat{y}_i) = \frac{1}{2}(y_i - \hat{y}_i)^2$

- Gradient $g_i = \frac{\partial L}{\partial \hat{y}_i} = -(y_i - \hat{y}_i) = -r_i$ (l'opposé du résidu).
- Hessienne $h_i = \frac{\partial^2 L}{\partial \hat{y}_i^2} = 1$ (constante).
- Somme des Hessiennes $\sum h_i = \sum 1 = N$.

Valeur Optimale : $w_j^* = \frac{\sum r_i}{N + \lambda}$

2. Cas de la Classification (Log-Loss)

Formule : $L(y_i, \hat{y}_i) = -[y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)]$

- Gradient $g_i = \dots = -(y_i - \hat{y}_i) = -r_i$.
- Hessienne $h_i = \dots = \hat{y}_i(1 - \hat{y}_i)$.
- Cette Hessienne correspond exactement au **Cover** !

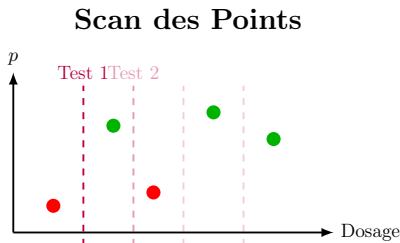
Valeur Optimale : $w_j^* = \frac{\sum r_i}{\sum \text{Cover}_i + \lambda}$

Algorithme Glouton (Exact Greedy Split)

Principe

L'algorithme parcourt **toutes** les valeurs triées de chaque caractéristique pour trouver le split optimal.

- Pour chaque variable, on trie les données.
- On scanne de gauche à droite.
- On calcule le Gain à chaque intervalle.



Scan de toutes les coupures possibles

Limitation

Extrêmement précis mais **coûteux** en mémoire et en temps sur des jeux de données massifs (Big Data).

Histogram-based Split Finding — Illustration

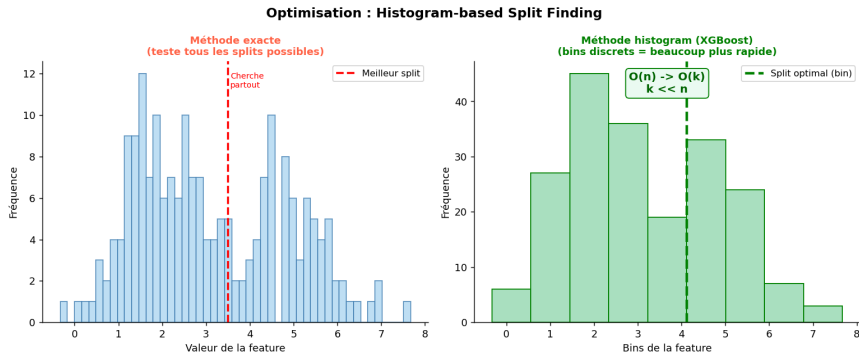


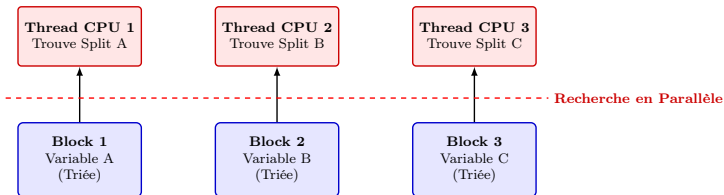
Figure 14 – XGBoost remplace la recherche exacte par des bins discrets : $O(n) \rightarrow O(k)$ avec $k \ll n$.

Apprentissage Parallèle (Column Blocks)

Innovation : Stockage par Blocs de Colonnes

XGBoost stocke les données dans une structure en mémoire appelée **Block**.

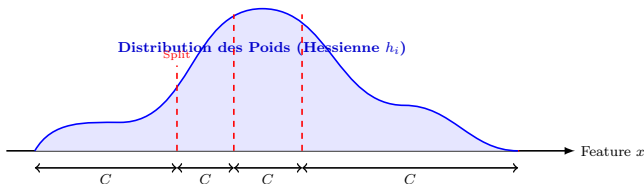
- Les données sont pré-triées par colonne avant l'entraînement.
- Chaque colonne peut être traitée par un cœur CPU différent **en parallèle**.
- Réduit drastiquement le coût du tri (l'étape la plus lente du boosting).



Innovation : Weighted Quantile Sketch

XGBoost ne découpe pas les "buckets" de manière uniforme. Il utilise la **Hessienne** (h_i) comme poids.

- Rappel : En classification, $h_i = p_i(1 - p_i)$ est la **confiance** ou la variance.
- Les points avec une forte Hessienne (incertitude) méritent des bacs plus précis.
- **Sketch** : Algorithme permettant de trouver ces quantiles sur des données massives avec peu de mémoire.



Les "buckets" sont plus étroits (plus de splits) là où l'incertitude (la Hessienne) est élevée.

Parcimonie (1/3) : Valeurs Manquantes

Réalité des données :

ID	Dosage	Effic. (y_i)	Résidu
1	2	0	-0.5
2	8	1	+0.5
3	NaN	1	+0.5
4	18	0	-0.5

Table 1 – $p_0 = 0.5 \implies r_i = y_i - 0.5$.
Donnée manquante pour l'ID 3.

Le défi

Les algorithmes classiques rejettent les lignes avec des valeurs manquantes ou nécessitent une imputation (*moyenne*, *mdiane*).

L'approche native de XGBoost

XGBoost traite la parcimonie (données vides, zéros, NaN) de manière **unifiée** et **apprise**.

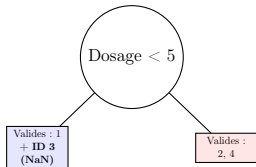
- **Aucune imputation nécessaire** : On ne "devine" pas la valeur.
- **Direction par défaut** : On apprend quel chemin (Gauche ou Droit) est le meilleur pour les données manquantes.

Parcimonie (2/4) : Test de la Coupure < 5

Étape 1 : Tester la coupure valide < 5

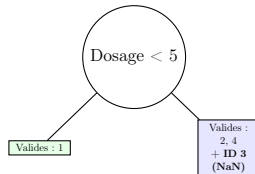
XGBoost calcule la pertinence de la coupure (Gain) sur les données observées, puis teste l'affectation du **NaN** à Gauche et à Droite.

Scénario A : NaN à GAUCHE



$$\text{Gain}_A = 0.00$$

Scénario B : NaN à DROITE



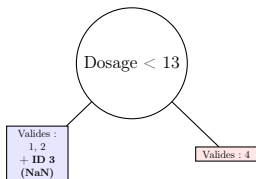
$$\text{Gain}_B = 1.33$$

Parcimonie (3/4) : Test de la Coupure < 13

Étape 2 : Tester l'autre coupure valide < 13

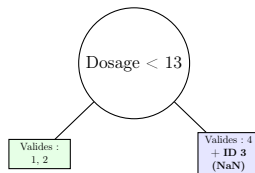
Il répète l'opération pour la coupure suivante (séparant ID 2 et 4).

Scénario C : NaN à GAUCHE



Gain_C = **1.33**

Scénario D : NaN à DROITE

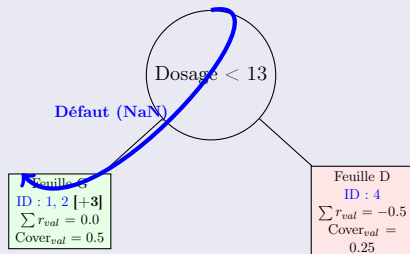


Gain_D = 0.00

Résultat de l'apprentissage :

- Le modèle a remarqué que les patients sans donnée de dosage (ID 3) ont souvent une efficacité positive ($y = 1$).
- La direction par défaut suivra donc le nœud qui mène aux prédictions positives.

Arbre avec directions par défaut

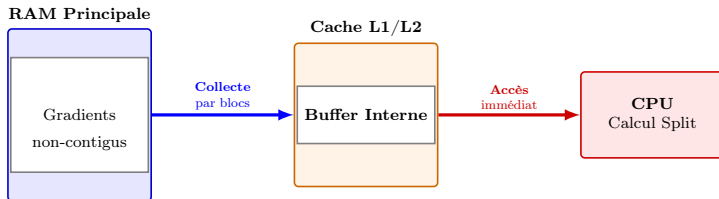


La flèche bleue indique le chemin automatique pour toute nouvelle donnée manquante.

Cache-Aware Prefetching

Les accès mémoire non contigus ralentissent le CPU. XGBoost utilise un **Buffer interne**.

- Les gradients sont récupérés par blocs pour tenir dans le **L1/L2 Cache**.
- Évite les "Cache Miss" (quand le CPU attend la RAM).
- Crucial pour la performance "Extreme" sur processeurs modernes.



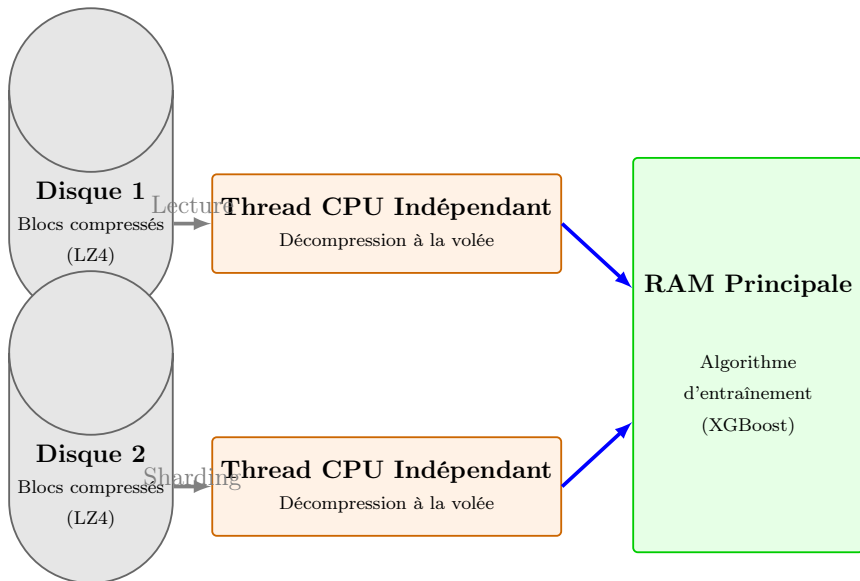
Évite au CPU d'attendre la lenteur de la RAM (Cache Miss)

Traitement du Big Data

Pour les données plus grandes que la RAM, XGBoost optimise les accès disque.

- **Compression par blocs** : Les données sur le disque sont compressées (algorithme LZ4 très performant).
- **Décompression à la volée** : Un thread indépendant du CPU s'occupe de la décompression avant que le modèle n'en ait besoin.
- **Sharding** : Les données sont réparties sur plusieurs disques pour paralléliser les lectures (I/O) et maximiser le débit.

Calculs Hors-Mémoire (Architecture)



Subsampling stochastique — Illustration

Subsampling stochastique dans XGBoost (subsample + colsample_bytree)

Dataset complet (lignes = obs., colonnes = features)

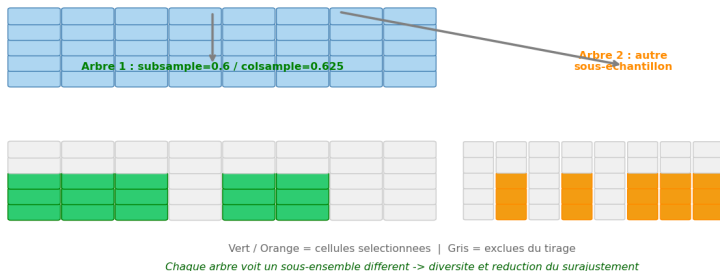


Figure 15 – Chaque arbre reçoit un sous-ensemble différent de lignes et colonnes.

Early Stopping — Illustration

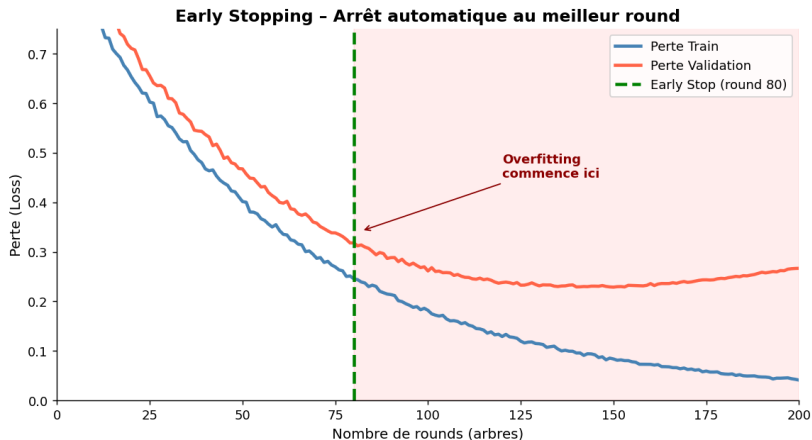


Figure 16 – L'entraînement s'arrête automatiquement au meilleur round de validation.

Régularisation XGBoost — Illustration

Effet de la régularisation XGBoost sur le surapprentissage

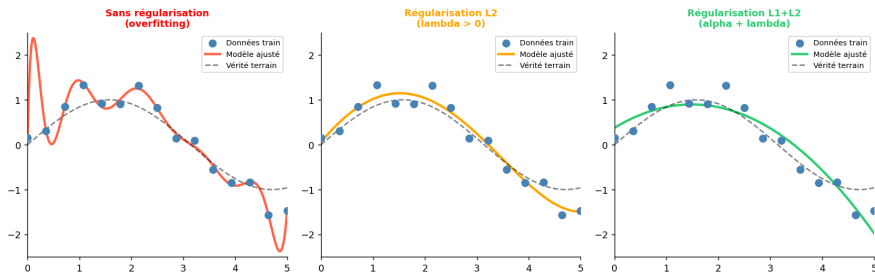


Figure 17 – La régularisation L1 (α) et L2 (λ) contrôle le surapprentissage.

Limitations du Gradient Boosting classique (GBM)

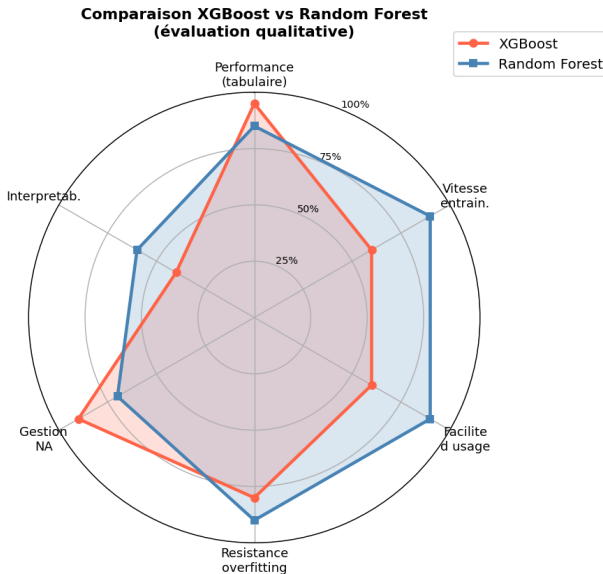
- Absence de régularisation explicite $\rightarrow$ risque élevé de surapprentissage
- Optimisation de premier ordre uniquement (gradient seul)
- Gestion limitée des valeurs manquantes
- Performances réduites sur grandes bases de données
- Manque d'optimisations systèmes pour le passage à l'échelle

- **Parallélisation** interne : recherche des meilleurs splits en parallèle
- Structure en colonnes (*Column Block*) pour accélérer les calculs
- Scalabilité sur grandes données :
 - **Histogram-based splits** (discrétisation en bins)
 - **Out-of-core computing** (gestion des données hors mémoire)
 - **Weighted Quantile Sketch** (approximation efficace des quantiles)

Hyperparamètres clés de XGBoost

- **Contrôle de la complexité :**
 - `max_depth`, `min_child_weight`, `gamma`
- **Régularisation et apprentissage :**
 - `lambda` (L2) et `alpha` (L1)
 - `learning_rate`
- **Sampling et autres :**
 - `subsample`, `colsample_bytree`
 - `scale_pos_weight`
 - `n_estimators` + `early_stopping_rounds`

Comparaison XGBoost vs Random Forest — Vue Radar



- XGBoost représente l'**état de l'art** du gradient boosting
- Trois piliers : **précision**, **efficacité** et **praticité**
- Optimisation de second ordre régularisée
- Parallélisation et gestion mémoire optimisées
- Fonctionnalités natives avancées
- Choix privilégié lorsque la **performance prédictive est critique**

- 1 Introduction
- 2 SVM
- 3 Bagging vs Boosting
- 4 Random Forests
- 5 XGBoost
- 6 Conclusion Générale**

Conclusion Générale

SVM

- Hyperplan à **marge maximale**
- Kernel Trick pour données non linéaires
- Optimum **global** garanti
- Sensible aux hyperparamètres (C, σ)
- Coûteux sur grands jeux de données

Random Forest

- **Bagging** d'arbres indépendants
- Réduction de la **variance** via agrégation
- Validation intégrée (**OOB**)
- Peu sensible aux hyperparamètres
- Ne peut pas **extrapoler**

XGBoost

- **Boosting** séquentiel régularisé
- Approximation de **Taylor 2nd ordre**
- Gestion native des **valeurs manquantes**
- Meilleure performance sur données tabulaires
- Réglage plus complexe (+20 hyperparamètres)

Application : Détection du Diabète (PIDD)

Les trois algorithmes ont été évalués sur la **Pima Indians Diabetes Database** (768 patientes, 8 variables cliniques, 35% de cas positifs).

XGBoost obtient les meilleures performances globales (AUC-ROC, F1-score).

Random Forest excelle en robustesse et rappel.

SVM (RBF) est compétitif mais sensible au déséquilibre des classes.

Variable clé :

La **glycémie plasmatique** est systématiquement le prédicteur le plus informatif dans les trois modèles.

Merci pour votre attention !

Des questions ?